Exploration in RL

Count-based Exploration

extrinsic reward r_t^{ext} – reward from env

intrinsic reward r_t^{int} – exploration bonus

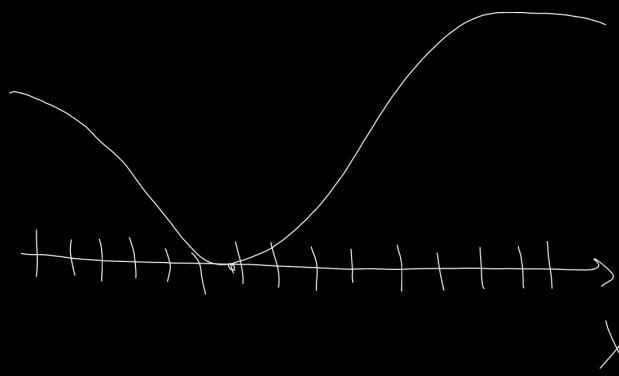
total reward $r_t = r_t^{ext} + \beta r_t^{int}$

Counter $N(s)$ or $N(s,a)$

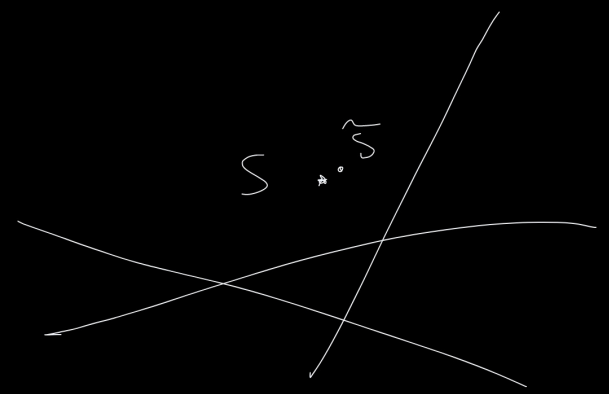
$$r_t^{int} = \frac{1}{\sqrt{N(s)+1}}.$$

Mountain Car $s \in \mathbb{R}^2$ – coordinate/velocity

Counter for each bin



Sim Hash



$s \in R^d$, $A \in R^{k \times d}$ – random and fixed matrix

$hash = sign(As) \in \{-1, 1\}^k$

Counter – dictionary on hashes

s – image, $\varphi(s)$ – embedding from autoencoder

$h = sign(A\varphi(s))$.

Pseudo-Counts

Generative model $p(s)$

PixelCNN, PixelRNN, Normalizing Flows

$$p(s) = \frac{N(s)}{N_{total}}, \quad p'(s) = \frac{N(s)+1}{N_{total}+1}$$

$$N(s) = p(s) N_{total}$$

$$p'(s) = \frac{p(s) N_{total} + 1}{N_{total} + 1} \Rightarrow$$

$$p'(s)(N_{total} + 1) = p(s) N_{total} + 1 \Rightarrow$$

$$N_{total} = \frac{1 - p'(s)}{p'(s) - p(s)}$$

$$N(s) = p(s) N_{total} = \frac{p(s)(1 - p'(s))}{p'(s) - p(s)}$$

$$\tau_t^{int} = \frac{1}{\sqrt{N(s) + 1}}$$

Adding r_t^{int} to RL agent

Off-policy RL (DQN, DDPG, SAC):

- use total reward $r_t = r_t^{ext} + \beta r_t^{int}$ for targets y_t for training Q functions
- when ~ replay buffer, we compute r_t^{int} anew

On-policy RL (PPO):

- use separate heads $V^{ext}(s)$, $V^{int}(s)$
- compute GAE^{ext}, GAE^{int} with different γ^{ext}, γ^{int}
- use $GAE^{ext} + \beta GAE^{int}$ for updating $\pi(a|s)$

Intrinsic Curiosity Module (ICM)

$s \longrightarrow \varphi(s) \in \mathbb{R}^d$ - embedding

s, a, s' $\quad a \approx \text{inverse_dynamic}(\varphi(s), \varphi(s') | \theta)$

$\text{Loss}_{inv} = \| a - \text{inverse_dynamic}(\varphi(s), \varphi(s') | \theta) \|^2$

or cross entropy loss for discrete action

$\varphi(s') \approx \text{forward_dynamic}(\varphi(s), a | \theta)$

$$Loss_{fwd} = \| \varphi(s') - forward_dyn(\varphi(s), a | \theta) \|^2$$

- $$Loss = \lambda Loss_{inv} + (1-\lambda) Loss_{fwd} \rightarrow \min_{\theta}$$

$$r_t^{int} = \| \varphi(s') - forward_dyn(\varphi(s), a | \theta) \|^2$$

Random Network Distillation (RND)

target network $f_{target}: S \to \mathbb{R}^k$ — Spectral normalization:

↑ random and fixed weights $W_{norm} = \frac{W}{\sigma_{max}(W)}$

prediction network $f_{pred}: S \to \mathbb{R}^k$

$$Loss = \| f_{target}(s) - f_{pred}(s|\theta) \|^2 \to \min_{\theta}$$

$$r_t^{int} = \| f_{target}(s) - f_{pred}(s) \|^2$$

..

$$r_{t,\,norm}^{int} = \frac{r_t^{int}}{\sigma + \varepsilon}$$, σ – running std.

Normalization of inputs in RL

Image s => divide each pixel by 255

$s \in \mathbb{R}^d$ => $s_{norm,i} = \frac{s_i - \mu_{running,i}}{\sigma_{running,i} + \varepsilon}$

Go-Explore

1. Exploration
2. Robustification

Archive of states

Choose state from archive → Go to the state → Explore from state → Update archive

random exploration

State → Cell : - domain-specific features
(x, y, room_id, num_keys)
- downsampled image

· Weight of a cell:

: $w(c) = f(\text{Count}(c), \text{Score}(c), \text{Neighbor}(c))$

Archive update: new cell → add to archive
old cell → update if cell has higher score

Robustification:

- get good trajectory
- init. s_{T-100}, learn PPO agent
- init. s_{T-200}, learn PPO agent